NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 1: Course Launch, Computation, Python Basics, Variables, and Types

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Problem ⟶ **Steps** ⟶ **Code** ⟶ **Evidence** ⟶ **Meaning**

About these notes

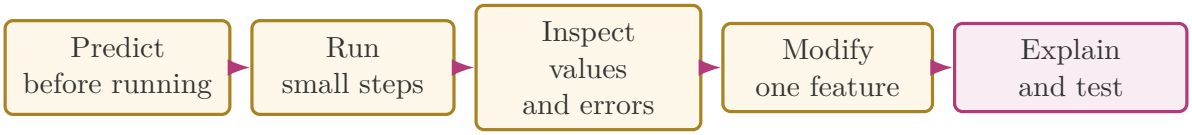
These reading notes expand the complete Week 1 lecture-slide set for *ENG 101 - Programming for Engineers*. The three lectures introduce the course, computational thinking, basic Python statements, mathematical operations, variables, and fundamental data types. The notes preserve the sequence and engineering emphasis of the slides while providing fuller explanations, annotated examples, worked calculations, self-check questions, and concise reference tables.

Every Python sample in these notes contains comments. The comments identify the purpose of the code, the physical units of engineering quantities, the intended calculation, or the reason a deliberately faulty example fails. Students should still run the examples themselves: reading code is valuable, but programming competence develops through prediction, execution, modification, testing, and explanation.

The course is designed for first-year engineering students and assumes no prior programming experience. It introduces programming through practical engineering problems and emphasizes problem solving, program design, coding, debugging, documentation, and communication.

How to use these notes

Programming cannot be learned effectively by reading passively. Use each worked example as a small experiment:



A productive study session follows five steps:

1. **Predict.** Before running a code fragment, write down the output or error you expect.
2. **Run.** Execute only a small, understandable section of code.
3. **Inspect.** Compare the actual behavior with your prediction. Read any error message carefully.
4. **Modify.** Change one value, unit, variable name, expression, or output format. Predict the effect first.
5. **Explain and test.** State what the program calculates, how you checked it, and what the result means physically.

Engineering check

A program that runs is not automatically correct. A trustworthy engineering calculation requires a suitable model, compatible units, correct implementation, planned tests, and a physically meaningful interpretation.

Week 1 structure

Lecture	Main theme	Core ideas
Lecture 1	Course launch and computational thinking	Course expectations; why engineers program; computation; algorithms; pseudocode; programs; the engineering-computation cycle.
Lecture 2	Python basics and mathematical operations	Python environments; statements and expressions; output; comments; error classes; arithmetic operators; precedence; scientific notation; units.
Lecture 3	Variables and data types	Assignment and reassignment; meaningful names; integers, floats, strings, Booleans, and `None`; input and conversion; floating-point approximation; integrated stress calculator.

Week 1 learning outcomes

By the end of Week 1, you should be able to:

1. explain why computation and programming matter in engineering;
2. distinguish an engineering problem, an algorithm, a program, and a numerical result;
3. run short Python statements and use Python as a calculator;
4. translate elementary mathematical formulas into Python expressions;
5. create meaningful variables and identify common Python data types;
6. keep physical units visible and perform basic unit conversions;
7. recognize syntax, runtime, logic, type, and unit-related problems;
8. test a simple engineering program using known cases and trend checks;
9. interpret a numerical output rather than treating it as a complete engineering conclusion.

Key idea

The main goal is not to memorize Python commands. The goal is to develop a repeatable way to transform an engineering question into a computational result that can be checked, interpreted, and communicated.

Contents

How to use these notes 3
Week 1 structure . . . 3
Week 1 learning outcomes . . . 4

1 Course Launch and Computational Thinking 7
1.1 Welcome to Programming for Engineers . . . 7
1.1.1 What ENG 101 is designed to do . . . 7
1.1.2 The semester journey . . . 8
1.2 Assessment, workload, and responsibility . . . 8
1.3 Why engineers program . . . 9
1.4 Opening engineering problem: energy use and cost . . . 9
1.4.1 Predicting a trend . . . 10
1.5 What a computer does - and does not do . . . 11
1.6 What is computation? . . . 11
1.7 Declarative and imperative knowledge . . . 11
1.7.1 Declarative knowledge: what . . . 12
1.7.2 Imperative knowledge: how . . . 12
1.8 Algorithms, pseudocode, and programs . . . 12
1.8.1 What makes a useful algorithm? . . . 12
1.8.2 Pseudocode . . . 13
1.8.3 From pseudocode to an annotated program . . . 13
1.9 The engineering-computation cycle . . . 14
1.10 Your first Python instruction . . . 15
1.10.1 A deliberate syntax error . . . 15
1.11 Lecture 1 summary . . . 15
1.12 Check your understanding . . . 16

2 Python Basics and Mathematical Operations 17
2.1 Why Python is a useful first engineering language . . . 17
2.2 Where Python code runs . . . 17
2.3 Statements, expressions, values, and output . . . 18
2.4 Comments document engineering meaning . . . 19
2.5 Three kinds of trouble . . . 20
2.5.1 Syntax error . . . 20
2.5.2 Runtime error . . . 20
2.5.3 Logic or unit error . . . 20
2.6 Python as a calculator . . . 21
2.6.1 Predict, run, and explain . . . 21
2.7 Why `/`, `//`, and `%` all exist . . . 22
2.8 Order of operations . . . 23
2.8.1 Kinetic energy . . . 23
2.8.2 Average of three measurements . . . 23
2.9 Translate the formula, not just the symbols . . . 24
2.10 Scientific notation and readable numbers . . . 24
2.11 Python stores numbers, not physical units . . . 25
2.12 Live coding: energy and cost calculator . . . 25
2.12.1 Modification experiments . . . 26
2.13 Debugging the heater program . . . 27

2.14 Lecture 2 summary . . . 28
2.15 Check your understanding . . . 28

3 Variables and Data Types 30
3.1 Meaningful names make engineering code auditable . . . 30
3.2 A variable is a name associated with an object . . . 31
3.3 Assignment is not algebraic equality . . . 31
3.3.1 Tracing reassignment . . . 32
3.4 Rules and conventions for variable names . . . 32
3.4.1 Units in variable names . . . 33
3.5 Common Python data types . . . 34
3.5.1 Inspecting types with `type()` . . . 34
3.5.2 Integers and floating-point numbers . . . 35
3.6 Types change the meaning of operators . . . 35
3.7 Boolean values represent engineering questions . . . 36
3.8 Input arrives as text . . . 36
3.9 A first floating-point surprise . . . 37
3.10 Integrated example: stress from force and area . . . 38
3.10.1 Assumption and interpretation . . . 39
3.10.2 Sanity trends . . . 39
3.11 Bug hunt: name, type, unit, and output . . . 39
3.12 Testing with simple cases before realistic ones . . . 40
3.12.1 Known-case test . . . 40
3.12.2 Trend tests . . . 41
3.13 The Week 1 concept map . . . 42
3.14 Lecture 3 summary . . . 42
3.15 Week 1 exit-ticket problems . . . 42

4 Week 1 Synthesis, Quick Reference, and Practice 44
4.1 The central Week 1 workflow . . . 44
4.2 Week 1 Python quick reference . . . 44
4.3 Engineering habits to start immediately . . . 45
4.4 Glossary . . . 45
4.5 Cumulative practice . . . 47
4.5.1 Concept questions . . . 47
4.5.2 Code tracing . . . 47
4.5.3 Formula translation . . . 48
4.5.4 Debugging practice . . . 48
4.5.5 Engineering interpretation . . . 48
4.6 Practice solutions . . . 49
4.6.1 Concept responses . . . 49
4.6.2 Code-tracing solutions . . . 49
4.6.3 Formula translations . . . 49
4.6.4 Debugging solution . . . 50
4.6.5 Interpretation response . . . 50

Source Basis and Suggested Reading 51

1 Course Launch and Computational Thinking

Lecture-slide alignment: Lecture 1 of the Week 1 slide set: why engineers program, what computation means, and how an algorithm becomes a program.

Learning outcomes

By the end of this chapter, you should be able to:

1. describe the purpose, assessment structure, and expected workload of ENG 101;
2. explain several ways programming supports engineering work;
3. define computation, algorithm, pseudocode, and program;
4. distinguish declarative knowledge from imperative knowledge;
5. convert a simple engineering problem into inputs, assumptions, formulas, pseudocode, code, tests, and interpretation;
6. run and explain a first Python `print()` statement.

1.1 Welcome to Programming for Engineers

Students begin ENG 101 with very different levels of experience. Some have never written a line of code; others have experimented with programming or completed an earlier course. This variation is normal. Week 1 is therefore not a test of speed or prior exposure. It is the beginning of a common way of thinking.

No previous programming experience is assumed. The habits that matter most at this stage are:

- curiosity about how a calculation can be expressed as a sequence of steps;
- willingness to try short programs and inspect what happens;
- persistence when a program produces an error;
- care with variable names, units, and formulas;
- readiness to explain a result rather than merely display it.

Key idea

An error message is not evidence that you are unsuited to programming. It is information about the difference between what Python can interpret and what you intended to express.

1.1.1 What ENG 101 is designed to do

ENG 101 is an introduction to programming for practical engineering problems. The course develops several connected abilities:

Problem solving
Turn an engineering question into a sequence of manageable computational tasks.

Programming
Express those tasks precisely in a high-level language.

Debugging
Find and repair mistakes in syntax, logic, data handling, and units.

Testing
Build evidence that a program behaves as intended.

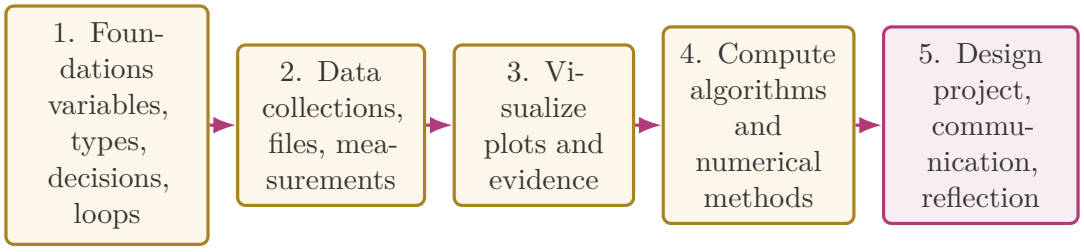
Communication
Make calculations readable, reproducible, and understandable to another person.

Transfer
Carry the underlying ideas into later work with MATLAB, C/C++, Julia, and other tools.

The course is not a typing competition and is not a catalogue of commands to memorize. It is also not a replacement for mathematics, physics, or engineering judgment. Python can evaluate the model you provide, but it does not automatically decide whether that model is physically suitable.

1.1.2 The semester journey

Week 1 establishes foundations that support the rest of the semester. The learning progression can be viewed as five connected stages.



Python is the first programming language used in the course, but the lasting skills are more general: decomposition, abstraction, data representation, algorithm design, testing, interpretation, and communication.

1.2 Assessment, workload, and responsibility

The course assessment weights are:

Assessment component	Weight
Final examination	40%
Midterm examination	30%
Laboratories	20%
Assignments	10%
Total	100%

Assessments may ask you to read, trace, write, modify, debug, test, and explain code. A

final program file is therefore only one form of evidence. Correctness, clarity, unit consistency, reasoning, and individual understanding all matter.

The course carries 6 ECTS and corresponds to 150 hours of student work: 45 contact hours and 105 hours of self-study. Students should plan for approximately 10–12 hours of total course work per week, including lectures, practice, laboratory work, assignments, and review.

A productive weekly rhythm is:

- **Before class:** skim the assigned material and attempt the opening question.
- **During class:** predict, discuss, run, modify, and explain short programs.
- **After class:** reproduce the examples from a clean session, complete practice, and record unresolved questions.

Collaboration can be valuable when it is permitted. Discussing concepts and comparing approaches can improve understanding. Submitted work, however, must reflect your own understanding and must follow the stated rules for collaboration, AI tools, and academic integrity for each assessment.

1.3 Why engineers program

Engineering involves more than substituting numbers into formulas. Engineers must analyze data, repeat calculations, compare alternatives, simulate systems, check constraints, and communicate evidence. Programming supports these activities in several ways.

Purpose	How programming helps
Calculate	Repeats formulas reliably, performs unit conversions, and creates design tables.
Analyze	Processes many measurements and helps identify trends, outliers, and unusual values.
Model	Represents a physical system and explores how its behavior changes with parameters or time.
Compare	Evaluates design alternatives, constraints, costs, and trade-offs.
Communicate	Produces reproducible calculations, reports, plots, and evidence.
Automate	Lets a computer handle scale and repetition while the engineer retains judgment and responsibility.

A one-line calculation may not need a program. The value of programming becomes clearer when the same calculation must be repeated, modified, checked, or shared.

1.4 Opening engineering problem: energy use and cost

Consider a workshop heater rated at 1,500 W. It operates for 2 h, and electricity costs \$0.12 per kW h. We want to determine:

1. the energy consumed in kW h;

2. the operating cost;
3. how the energy and cost change if the operating time becomes 5 h.

Before using Python, identify the inputs and units:

Quantity	Symbol	Value and unit
Power rating	P	1,500 W
Operating time	t	2 h
Electricity price	c	\$0.12 per kW h

Electricity energy in kilowatt-hours is calculated from

$$E_{\mathrm{kWh}} = P_{\mathrm{kW}} t_{\mathrm{h}},$$

and cost is

$$C = E_{\mathrm{kWh}} c.$$

The power must first be converted:

$$1{,}500\,\mathrm{W} = 1.5\,\mathrm{kW}.$$

Therefore,

$$E_{\mathrm{kWh}} = (1.5\,\mathrm{kW})(2\,\mathrm{h}) = 3\,\mathrm{kW\,h},$$

and

$$C = (3\,\mathrm{kW\,h})(\$0.12/\mathrm{kW\,h}) = \$0.36.$$

Engineering check

The unit kW h is an energy unit. It is obtained by multiplying power in kilowatts by time in hours. Multiplying watts directly by hours does not produce kilowatt-hours unless the watt value is first divided by 1000.

1.4.1 Predicting a trend

If time changes from 2 h to 5 h while power and price stay fixed, both energy and cost should be multiplied by

$$\frac{5}{2} = 2.5.$$

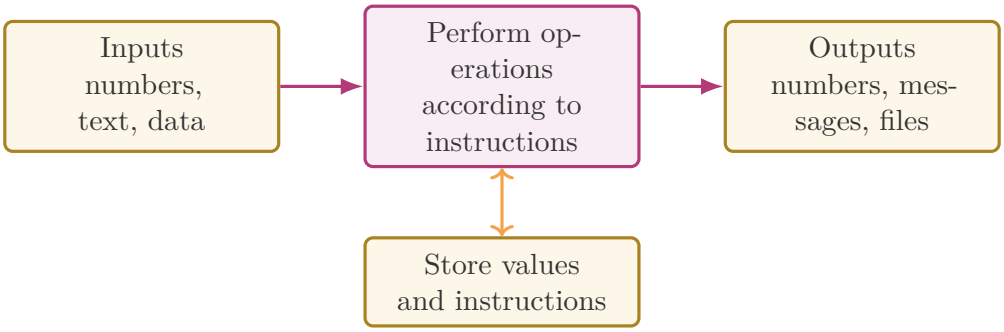
The expected new values are therefore

$$E = 7.5\,\mathrm{kW\,h}, \qquad C = \$0.90.$$

This prediction is not an optional extra. It provides evidence against which the later program can be checked. If the program predicts a decrease, no change, or a different scaling factor, the discrepancy must be investigated.

1.5 What a computer does - and does not do

A computer accepts inputs, stores values and instructions, performs operations according to those instructions, and produces outputs. The inputs and outputs may include numbers, text, files, images, or other data.



Computers are extraordinarily fast and can store large amounts of information, but speed and memory do not create meaning. The machine follows the instructions it receives. The engineer must decide whether the instructions, assumptions, data, and results are appropriate.

Key idea

A computer can calculate quickly. It cannot automatically know that a force is in newtons, an area is in square millimetres, or a model is unsuitable for the physical situation.

1.6 What is computation?

Definition

Computation is the transformation of inputs into outputs by following a well-defined sequence of steps.

In engineering, inputs and outputs usually have physical meaning and units. For example, the normal-stress calculation

$$\sigma = \frac{F}{A}$$

uses force F and cross-sectional area A as inputs, evaluates a formula, and produces stress σ as output. The calculation is incomplete until we ask:

- Are the units compatible?
- What assumptions justify the formula?
- Is the magnitude reasonable?
- What does the result imply for the design or experiment?

1.7 Declarative and imperative knowledge

Computational thinking requires a distinction between knowing *what* something is and knowing *how* to calculate it.

1.7.1 Declarative knowledge: what

A mathematical definition of the nonnegative square root is

$$y = \sqrt{x} \quad \Longleftrightarrow \quad y^2 = x, \quad y \geq 0.$$

This statement tells us what a square root is. It does not, by itself, tell a computer how to find one.

1.7.2 Imperative knowledge: how

An imperative description gives a procedure:

1. choose a guess g;
2. calculate g^2 and compare it with x;
3. improve the guess;
4. repeat until the guess is accurate enough.

This description contains actions, an order, a repeated step, and a stopping condition. It can therefore become an algorithm. Later in the course, guess-and-check and bisection methods will turn this idea into executable programs.

Key idea

Computational thinking turns a desired relationship or outcome into a procedure that can be executed and checked.

1.8 Algorithms, pseudocode, and programs

1.8.1 What makes a useful algorithm?

Definition

An **algorithm** is a finite, ordered, and sufficiently precise sequence of instructions for transforming specified inputs into defined outputs.

A useful algorithm should satisfy the following properties:

1. Inputs are specified.
2. Each step is precise enough to follow without guessing.
3. The order of the steps is clear.
4. The process terminates for the intended cases.
5. Outputs are defined.
6. The method handles the cases it claims to handle.
7. The result can be checked.

An algorithm describes the method independently of a particular programming language. **Code**

expresses the algorithm precisely using the syntax and semantics of a programming language.

1.8.2 Pseudocode

Pseudocode is a structured description of an algorithm written in plain language. It is more precise than an informal idea but less restrictive than Python syntax.

For the heater problem, suitable pseudocode is:

Worked example: heater-calculator pseudocode

1. Store the heater power in watts.
2. Convert power from watts to kilowatts.
3. Store the operating time in hours.
4. Store the electricity price per kilowatt-hour.
5. Calculate energy in kilowatt-hours.
6. Calculate operating cost.
7. Display energy and cost with labels and units.
8. Check that the results agree with the hand calculation and expected trend.

Notice that the pseudocode contains a unit conversion and a check. An algorithm that says only "calculate cost" is not sufficiently precise.

1.8.3 From pseudocode to an annotated program

The following Python program implements the same sequence. Every line includes a comment explaining its purpose or unit.

```
# Store the heater power using watts as the input unit.
power_w = 1500

# Convert watts to kilowatts because electricity energy is required in kWh.
power_kw = power_w / 1000

# Store the operating time in hours.
time_h = 2

# Store the electricity price in dollars per kilowatt-hour.
price_per_kwh = 0.12

# Calculate energy: kilowatts multiplied by hours gives kilowatt-hours.
energy_kwh = power_kw * time_h

# Calculate cost: energy multiplied by the price per kilowatt-hour.
cost = energy_kwh * price_per_kwh

# Display the two calculated values; clearer formatting is added in Lecture 2.
print(energy_kwh)
print(cost)
```

Code 1.1: First annotated heater energy and cost program.

Expected output:

```
3.0
0.36
```

Code 1.2: Output from the first heater program.

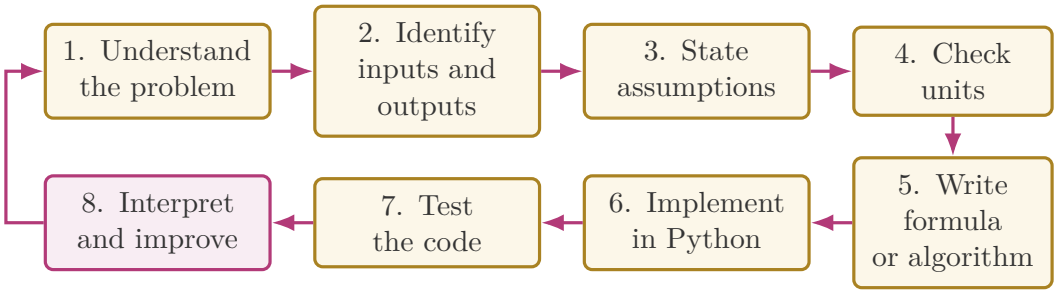
The numerical values are correct, but the output is not yet ideal. A reader sees `3.0` and `0.36` without labels or units. Lecture 2 improves the report using formatted strings.

Engineering check

A computer executes the program that was written, not the intention the programmer had in mind. Precision in variable names, units, formulas, and output is therefore essential.

1.9 The engineering-computation cycle

The course repeatedly uses the following cycle.



Each stage asks a different question:

Stage	Questions
Understand	What physical or design problem is being addressed? What exactly is being requested?
Inputs and outputs	What values are known? What must be calculated or reported?
Assumptions	What effects are ignored? Under what conditions is the model appropriate?
Units	Are the quantities expressed in compatible units? Are squared and cubed conversions handled correctly?
Formula or algorithm	Which relationship or ordered procedure connects the inputs to the outputs?
Implementation	How should the steps be represented as Python statements and expressions?
Testing	What known case, trend, boundary, or comparison can provide evidence of correctness?
Interpretation and improvement	What does the result mean physically? What limitations remain? How could the program become clearer or more robust?

Key idea

Syntax tells Python what to do. Engineering reasoning tells us what should be done, how it should be checked, and what the result means.

1.10 Your first Python instruction

A first Python statement can be only one line long.

```
# Call print() to display one string, which is a text value.
print("Welcome to Programming for Engineers")
```

Code 1.3: Displaying a welcome message.

Output:

```
Welcome to Programming for Engineers
```

Code 1.4: Output from the first print statement.

Read the instruction aloud: "Call the function `print` and give it one text value to display." The quotation marks tell Python that the enclosed characters form a string.

1.10.1 A deliberate syntax error

The following sample is intentionally incorrect.

```
# Incorrect: the string begins with a quotation mark but never closes.
print("Welcome to Programming for Engineers)
```

Code 1.5: Deliberately faulty code: missing closing quotation mark.

Python cannot determine where the string ends, so it reports a syntax error. The important first response is not to guess randomly. Read the error message, inspect the indicated line, and compare opening and closing punctuation.

Quotation marks, parentheses, and brackets usually occur in matching pairs. When Python reports invalid syntax near the end of a line, inspect the punctuation earlier on the same line as well.

1.11 Lecture 1 summary

Section summary

- ENG 101 teaches programming as engineering problem solving, not as command memorization.
- Engineers program to calculate, analyze, model, compare, communicate, and automate.
- Computation transforms meaningful inputs into outputs through well-defined steps.

- Declarative knowledge states what is true; imperative knowledge describes how to obtain a result.
- An algorithm is an ordered, finite, precise, and checkable procedure.
- Pseudocode helps separate problem understanding from Python syntax.
- The engineering-computation cycle includes assumptions, units, testing, and interpretation.
- A program may be very short, but its punctuation and meaning must still be precise.

1.12 Check your understanding

Answer these questions before moving to Lecture 2.

1. What is the difference between an algorithm and a program?
2. Why is the mathematical definition $y^2 = x$ not, by itself, an algorithm for finding $\sqrt{x}$?
3. What unit conversion is required before multiplying a heater power in watts by time in hours to obtain energy in kilowatt-hours?
4. If heater operating time doubles while all other quantities stay fixed, what should happen to energy and cost?
5. Name two decisions that remain the engineer's responsibility after Python produces a numerical result.
6. Explain why testing should be planned before realistic input values are used.

1. An algorithm describes a method; a program expresses that method in a programming language so a computer can execute it.
2. The definition states the required relationship but gives no ordered steps, update rule, or stopping condition for constructing the value.
3. Divide the watt value by 1000 to obtain kilowatts.
4. Both energy and cost should double because both are directly proportional to operating time in this model.
5. Examples include deciding whether the model is appropriate, whether the units are compatible, whether the magnitude is reasonable, and whether the result satisfies a design requirement.
6. A known prediction provides independent evidence. Without it, a plausible-looking result may be accepted even when the code or units are wrong.

2 Python Basics and Mathematical Operations

Lecture-slide alignment: Lecture 2 of the Week 1 slide set: running code, output, comments, errors, arithmetic operators, precedence, scientific notation, and units.

Learning outcomes

By the end of this chapter, you should be able to:

1. distinguish an interactive shell, notebook cell, and saved Python script;
2. explain the difference among a statement, an expression, a value, and displayed output;
3. use `print()` and comments in short programs;
4. recognize syntax, runtime, logic, and unit-related errors;
5. use `+`, `-`, `*`, `/`, `//`, `%`, and `**`;
6. apply operator precedence and parentheses when translating engineering formulas;
7. use scientific notation and digit-grouping underscores;
8. preserve units in variables, comments, formulas, prompts, and output;
9. implement, modify, and debug the heater energy and cost calculator.

2.1 Why Python is a useful first engineering language

Python is suitable for an introductory engineering course because it offers two complementary advantages.

Accessible now	Powerful later
Readable syntax	Numerical and scientific computing
Immediate feedback	Data analysis and visualization
Useful as an interactive calculator	Simulation and automation
Short programs can solve meaningful problems	Machine learning and extensive scientific libraries

The course does not attempt to cover every Python feature. Python is a vehicle for learning how to construct, test, and explain computational solutions. The same habits remain useful when the language changes.

2.2 Where Python code runs

Students may encounter Python in several environments.

Interactive shell

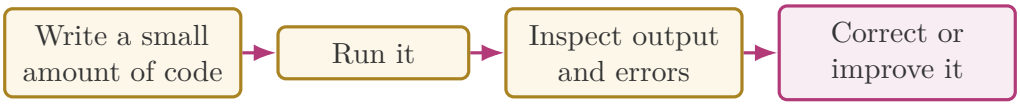
You enter one statement or expression and receive an immediate response. This is useful for

testing small ideas.

Notebook
Code is organized into cells, often alongside explanatory text and displayed output. Notebooks are useful for exploration, calculations, and reports, but cells should be run in a clear order.

Script or IDE
Code is saved in a file, usually with the extension `.py`. An integrated development environment provides editing, execution, and debugging tools.

The supported course environment is specified on Moodle; the course documentation identifies Anaconda and Spyder as supported software. Regardless of environment, the basic working habit is the same:



In a notebook, code may appear to work only because an earlier cell created a variable. Before submitting a notebook, restart the Python kernel and run all cells from the beginning. This reveals hidden dependencies and out-of-order execution.

2.3 Statements, expressions, values, and output

These words describe different parts of program behavior.

Definition

Statement
An instruction that tells Python to do something.

Expression
A combination of literals, variables, and operators that Python evaluates.

Value
The object produced by evaluating an expression.

Output
Information displayed or written by a program.

Consider the following fully annotated program.

```
# Display a string literal containing the course code.
print("ENG 101")

# Evaluate the arithmetic expression 3 + 2, then display its value.
print(3 + 2)

# Pass three values to print(); Python inserts spaces between them.
print("Voltage =", 12, "V")
```

Code 2.1: Statements, expressions, values, and printed output.

Expected output:

```
ENG 101
5
Voltage = 12 V
```

Code 2.2: Output from the statements-and-values example.

The expression `3 + 2` has the value `5`. The function `print()` displays that value, but the value and its display are not the same concept. A value can be stored, used in another calculation, returned from a function, or displayed in different formats.

2.4 Comments document engineering meaning

A Python comment begins with `#`. Python ignores the comment when executing the program, but a human reader uses it to understand units, assumptions, conversions, and limitations.

```
# Store speed in metres per second.
speed_m_per_s = 20

# Store elapsed time in seconds.
time_s = 15

# Calculate distance; (m/s) multiplied by s gives m.
distance_m = speed_m_per_s * time_s

# Display the result with one decimal place and the unit m.
print(f"Distance = {distance_m:.1f} m")
```

Code 2.3: Distance calculation with engineering comments.

Output:

```
Distance = 300.0 m
```

Code 2.4: Output from the distance calculation.

Useful comments explain:

- the units of an input or output;
- an assumption used by the model;
- why a conversion is necessary;
- a limitation, warning, or source of uncertainty;
- a non-obvious design decision in the code.

A good comment explains the reason for a step:

```
# Convert square millimetres to square metres before calculating stress.
area_m2 = area_mm2 / 1_000_000
```

Code 2.5: A useful unit-conversion comment.

A weaker comment merely repeats the symbols:

```
# Divide area_mm2 by one million.
area_m2 = area_mm2 / 1_000_000
```

Code 2.6: A less useful comment that only repeats the operation.

Both comments are technically correct, but the first helps a reader audit the engineering logic.

2.5 Three kinds of trouble

Errors can appear at different stages. Week 1 introduces three broad categories.

2.5.1 Syntax error

A syntax error occurs when Python cannot parse the structure of the program.

```
# Incorrect: the opening quotation mark has no matching closing mark.
print("Stress)
```

Code 2.7: Deliberate syntax error: the string is not closed.

Execution does not begin because Python cannot identify a legal statement.

2.5.2 Runtime error

A runtime error occurs after execution starts.

```
# Store an invalid cross-sectional area of zero square metres.
area_m2 = 0

# Runtime error: a finite force cannot be divided by zero area.
stress_pa = 5000 / area_m2
```

Code 2.8: Deliberate runtime error: division by zero.

Python can parse both statements, but the division fails when the second statement is executed.

2.5.3 Logic or unit error

A logic or unit error may allow the program to run to completion while producing a wrong engineering result.

```
# Incorrect: dividing by 1000 converts mm to m, not mm^2 to m^2.
area_m2 = area_mm2 / 1000
```

Code 2.9: Deliberate unit error: wrong conversion factor for area.

The correct conversion is

$$1\ \text{m}^2 = 1{,}000{,}000\ \text{mm}^2,$$

so square millimetres must be divided by one million.

Engineering check

The most dangerous error may be one that produces a plausible-looking output. A visible crash invites investigation; a silent unit error may be accepted and used in a design decision.

2.6 Python as a calculator

The arithmetic operators used in Week 1 are summarized below.

Operator	Meaning	Example
`+`	addition	`3 + 2` gives `5`
`-`	subtraction	`3 - 2` gives `1`
`*`	multiplication	`3 * 2` gives `6`
`/`	true division	`3 / 2` gives `1.5`
`//`	floor division	`7 // 3` gives `2`
`%`	remainder (modulo)	`7 \% 3` gives `1`
`**`	exponentiation	`3 ** 2` gives `9`

Python uses `**` for exponentiation. The symbol `^` is not the Python exponentiation operator.

2.6.1 Predict, run, and explain

Do not run the following program immediately. Predict the six outputs first.

```
# Multiplication is evaluated before addition.
print(5 + 2 * 3)

# Parentheses force addition to be evaluated first.
print((5 + 2) * 3)

# True division keeps the fractional part.
print(10 / 4)

# Floor division returns the quotient rounded down.
print(10 // 4)

# Modulo returns the remainder after whole-number grouping.
print(10 % 4)

# Exponentiation is right-associative: 3 ** 2 is evaluated first.
print(2 ** 3 ** 2)
```

Code 2.10: Operator-precedence prediction activity.

Output:

```
11
21
2.5
2
```

```
2
512
```

Code 2.11: Output from the operator-precedence activity.

The last expression is interpreted as

$$2^{(3^2)} = 2^9 = 512,$$

not as

$$(2^3)^2 = 8^2 = 64.$$

Use parentheses when you want the grouping to be unmistakable.

2.7 Why /, //, and % all exist

These three operators answer different questions.

True division: `17 / 5`
gives `3.4`. Use it when the fractional result matters.

Floor division: `17 // 5`
gives 3. It counts how many complete groups of five fit into seventeen.

Remainder: `17 \% 5`
gives 2. It reports how many items remain after forming complete groups.

Engineering uses include batching samples, scheduling periodic maintenance, separating total seconds into complete minutes and remaining seconds, and detecting periodic patterns.

Worked example: converting seconds to minutes and seconds

Suppose a test lasts 137 seconds. We can calculate the number of complete minutes and the remaining seconds.

```python
# Store the total duration in seconds.
total_seconds = 137

# Count complete groups of 60 seconds.
whole_minutes = total_seconds // 60

# Find the seconds left after removing complete minutes.
remaining_seconds = total_seconds % 60

# Display the decomposed duration.
print(f"Duration = {whole_minutes} min {remaining_seconds} s")
```

Code 2.12: Using floor division and remainder for time conversion.

```
Duration = 2 min 17 s
```

Code 2.13: Output from the time-conversion example.

2.8 Order of operations

Python follows the usual precedence rules:

1. parentheses;
2. exponentiation;
3. multiplication and division, including `*`, `/`, `//`, and `%`;
4. addition and subtraction.

Operations at the same precedence level are generally evaluated from left to right, except exponentiation, which is right-associative.

Key idea

Use parentheses whenever they make the engineering formula clearer, even if Python would produce the same result without them.

2.8.1 Kinetic energy

The formula

$$KE = \frac{1}{2}mv^2$$

can be translated directly.

```
# Store mass in kilograms.
mass_kg = 1200.0

# Store speed in metres per second.
velocity_m_per_s = 20.0

# Translate KE = (1/2) m v^2; Python uses ** for the square.
kinetic_energy_j = 0.5 * mass_kg * velocity_m_per_s**2

# Display kinetic energy in joules.
print(f"Kinetic energy = {kinetic_energy_j:.1f} J")
```

Code 2.14: Kinetic-energy expression with annotated inputs.

2.8.2 Average of three measurements

The arithmetic mean

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3}$$

requires parentheses around the numerator.

```
# Store three measurements in the same physical unit.
x1 = 10.2
x2 = 10.5
x3 = 10.4

# Add all measurements before dividing the total by three.
average = (x1 + x2 + x3) / 3
```

```
8
9  # Display the mean to two decimal places.
10 print(f"Average = {average:.2f}")
```

Code 2.15: Average of three measurements.

Without the parentheses, only `x3` would be divided by 3 before the other values were added.

2.9 Translate the formula, not just the symbols

A Python expression should preserve the mathematical structure and the physical meaning of the formula.

Mathematics	Annotated Python translation
$\bar{x} = \dfrac{x_1 + x_2 + x_3}{3}$	`average = (x1 + x2 + x3) / 3` The parentheses preserve the complete numerator.
$P = \dfrac{V^2}{R}$	`power_w = voltage_v**2 / resistance_ohm` The square is applied to voltage before division by resistance.
$\sigma = \dfrac{F}{A}$	`stress_pa = force_n / area_m2` Force and area must use compatible SI units.

When translating a formula, check every numerator, denominator, exponent, and parenthesis. Then check the units.

Write the correct Python expression for

$$y = \frac{a + b}{c^2}.$$

A clear answer is `y = (a + b) / c**2`. The parentheses ensure that the whole numerator is divided by c^2.

2.10 Scientific notation and readable numbers

Engineering quantities may be extremely large or small. Python uses `e` notation for powers of ten.

```
1  # Store 250 million pascals using scientific notation.
2  yield_strength_pa = 250e6
3
4  # Store 0.0004 square metres using scientific notation.
5  area_m2 = 4.0e-4
6
7  # Store an approximate speed of light in metres per second.
8  speed_of_light_m_per_s = 3.0e8
9
10 # Use underscores to make a large integer easier for humans to read.
```

```
one_million = 1_000_000
```

Code 2.16: Scientific notation and digit-grouping underscores.

The interpretations are

$$250e6 = 250 \times 10^{6}, \qquad 4.0e{-}4 = 4.0 \times 10^{-4}.$$

Underscores do not change a number's value. They improve readability.

```
# Store stress in pascals.
stress_pa = 125e6

# Divide by one million because 1 MPa = 1,000,000 Pa.
stress_mpa = stress_pa / 1e6

# Display the converted stress.
print(f"Stress = {stress_mpa:.1f} MPa")
```

Code 2.17: Converting stress from pascals to megapascals.

The program prints `125.0 MPa`.

2.11 Python stores numbers, not physical units

The number `400` could mean $400\,\text{mm}^2$, $400\,\text{m}^2$, $400\,\text{Pa}$, or something else. Python stores the numerical object; engineering meaning comes from the variable name, comments, prompt, formula, and output.

Four habits help preserve units:

1. put units in comments;
2. include units in variable names when useful;
3. state expected units in input prompts;
4. include units in printed output.

A variable named only `area` does not reveal whether its value is in mm^2 or m^2. When both units appear in one program, names such as `area_mm2` and `area_m2` make the conversion auditable.

2.12 Live coding: energy and cost calculator

The first heater program is now improved with comments, clear variable names, and formatted output.

```
# Store the heater rating in watts.
power_w = 1500

# Convert watts to kilowatts before calculating energy in kWh.
```

```
power_kw = power_w / 1000

# Store operating time in hours.
time_h = 2

# Store the electricity price in dollars per kilowatt-hour.
price_per_kwh = 0.12

# Calculate energy; kW multiplied by h gives kWh.
energy_kwh = power_kw * time_h

# Calculate operating cost in dollars.
cost = energy_kwh * price_per_kwh

# Display energy with two digits after the decimal point and a unit label.
print(f"Energy used = {energy_kwh:.2f} kWh")

# Display cost with a dollar sign and two digits after the decimal point.
print(f"Cost = ${cost:.2f}")
```

Code 2.18: Complete annotated energy and cost calculator.

Expected output:

```
Energy used = 3.00 kWh
Cost = $0.36
```

Code 2.19: Output from the complete heater calculator.

The result agrees with the prediction from Lecture 1. This agreement is evidence, although it is not the only possible test.

2.12.1 Modification experiments

The slides ask students to modify the program in four ways. The expected results are:

1. **Time changes from 2 h to 5 h.** Energy becomes 7.5 kW h; cost becomes \$0.90.
2. **Power changes from 1500 W to 2000 W.** For 2 h, energy becomes 4 kW h; cost becomes \$0.48.
3. **Electricity price becomes zero.** Energy is unchanged, but cost becomes \$0.00.
4. **Power doubles while time is halved.** The product Pt is unchanged, so energy and cost remain unchanged.

A commented version for the fourth experiment is:

```
# Double the original 1500 W heater power.
power_w = 3000

# Convert the doubled power to kilowatts.
power_kw = power_w / 1000

# Halve the original two-hour operating time.
time_h = 1

```

```
# Keep the electricity price unchanged.
price_per_kwh = 0.12

# Calculate energy; doubled power and halved time keep the product unchanged.
energy_kwh = power_kw * time_h

# Calculate the corresponding cost.
cost = energy_kwh * price_per_kwh

# Display the result for comparison with the original case.
print(f"Energy used = {energy_kwh:.2f} kWh")
print(f"Cost = ${cost:.2f}")
```

Code 2.20: Testing a change that keeps the product of power and time constant.

2.13 Debugging the heater program

Consider this deliberately faulty version.

```
# Store power in watts.
power_w = 1500

# Store time in hours.
time_h = 2

# ERROR: W multiplied by h is not kWh; power must first be converted to kW.
energy_kwh = power_w * time_h

# This line uses the incorrectly scaled energy value.
cost = energy_kwh * 0.12

# ERROR: quotation marks make energy_kwh literal text rather than a variable value.
print("Energy = energy_kwh kWh")

# The f-string works syntactically, but cost is wrong because energy is wrong.
print(f"Cost = ${cost:.2f}")
```

Code 2.21: Faulty heater program with comments identifying the intended roles.

There are two central problems:

1. **Unit error.** The program multiplies watts by hours but labels the result as kilowatt-hours. The power must be divided by 1000 first.
2. **Output error.** The string `"Energy = energy_kwh kWh"` displays the letters `energy_kwh`, not the variable's value. An f-string is needed.

Corrected version:

```
# Store heater power in watts.
power_w = 1500

# Convert watts to kilowatts for a kWh calculation.
power_kw = power_w / 1000

```

```
# Store operating time in hours.
time_h = 2

# Calculate energy in kilowatt-hours.
energy_kwh = power_kw * time_h

# Calculate cost using dollars per kilowatt-hour.
cost = energy_kwh * 0.12

# Use an f-string so the value of energy_kwh is inserted into the report.
print(f"Energy = {energy_kwh:.2f} kWh")

# Display cost with two digits after the decimal point.
print(f"Cost = ${cost:.2f}")
```

Code 2.22: Corrected and fully annotated heater program.

2.14 Lecture 2 summary

Section summary

- Python code may run in a shell, notebook, or saved script, but the core cycle is write, run, inspect, and correct.
- A statement instructs Python; an expression is evaluated; a value is produced; `print()` displays values.
- Comments document units, assumptions, conversions, and limitations.
- Syntax errors prevent parsing, runtime errors stop execution, and logic or unit errors may silently produce wrong results.
- Python uses `**` for powers, `/` for true division, `//` for floor division, and `%` for the remainder.
- Parentheses should make the structure of an engineering formula explicit.
- Scientific notation and underscores make extreme or large values readable.
- Python does not automatically attach physical units to numbers.
- The heater calculator demonstrates prediction, unit conversion, formatted output, modification, and debugging.

2.15 Check your understanding

1. Explain the difference between `10 / 4`, `10 // 4`, and `10 \% 4`.
2. Predict the value of `2 ** 3 ** 2` and explain the grouping.
3. Translate $P = V^2/R$ into a readable Python expression using variables with units in their names.
4. Why is `area_mm2 / 1000` not a correct conversion to square metres?

5. What is the difference between `print("stress_pa")` and `print(stress_pa)`?
6. Classify each problem as syntax, runtime, or logic/unit related:
   (a) a missing quotation mark;
   (b) division by zero;
   (c) labeling watt-hours as kilowatt-hours.

1. True division gives `2.5`; floor division gives `2`; remainder gives `2`.
2. It gives `512` because exponentiation is right-associative: $2^{(3^2)} = 2^9$.
3. `power_w = voltage_v**2 / resistance_ohm`.
4. A linear conversion factor must be squared for area: $1000^2 = 1{,}000{,}000$.
5. The first prints literal text; the second evaluates the variable and prints its value.
6. (a) syntax; (b) runtime; (c) logic/unit.

3 Variables and Data Types

Lecture-slide alignment: Lecture 3 of the Week 1 slide set: assignment, meaningful names, data types, input conversion, floating-point behavior, testing, and an integrated stress calculation.

Learning outcomes

By the end of this chapter, you should be able to:

1. explain a variable as a name associated with a Python object;
2. distinguish assignment from algebraic equality;
3. trace reassignment step by step;
4. choose valid, meaningful variable names that keep units visible;
5. identify `int`, `float`, `str`, `bool`, and `None`;
6. use `type()`, `int()`, `float()`, and `str()`;
7. explain why operators may behave differently for different data types;
8. recognize a basic floating-point representation effect;
9. build, debug, test, and interpret a simple engineering stress program.

3.1 Meaningful names make engineering code auditable

Compare two versions of the same calculation.

```
# Store a numerical input, but the name a does not reveal what it represents.
a = 12.0

# Store a second input, but the name b does not reveal its unit or role.
b = 4.0

# Divide the two values, but the name c does not describe the result.
c = a / b
```

Code 3.1: A calculation with names that hide the engineering meaning.

This program may run correctly, but another person cannot easily determine what the values mean. Now compare:

```
# Store the supply voltage in volts.
voltage_v = 12.0

# Store electrical resistance in ohms.
resistance_ohm = 4.0

# Apply Ohm's law I = V / R; the result is current in amperes.
current_a = voltage_v / resistance_ohm
```

Code 3.2: Ohm's-law calculation with meaningful names and visible units.

The second version exposes the relationship

$$I = \frac{V}{R}.$$

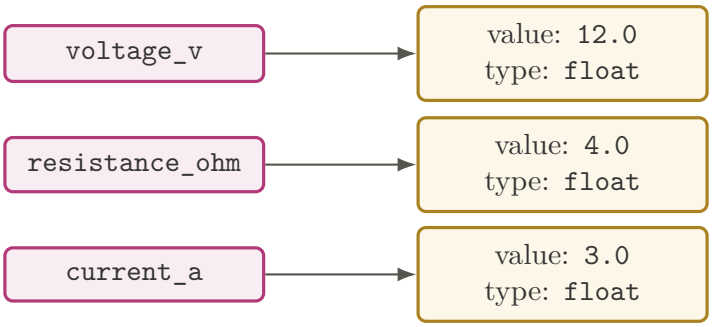
Names are therefore part of technical communication. They help the original programmer, teammates, instructors, reviewers, and future users understand and audit the calculation.

Key idea

A good variable name describes the quantity, not merely its position in a calculation.

3.2 A variable is a name associated with an object

Python manipulates objects. Each object has a value and a type. A variable is a name associated with an object.



A beginner may imagine a variable as a permanent box containing a number. That model becomes misleading when values are reassigned or when several names refer to objects. A more useful mental model is:

> Assignment associates, or re-associates, a name with the object produced by evaluating the expression on the right-hand side.

3.3 Assignment is not algebraic equality

In algebra, the statement

$$x = x + 1$$

has no finite solution. In Python, the symbol `=` is an assignment operator, not an equality claim.

```
# Associate the name x with the integer object 4.
x = 4

# Evaluate x + 1 using the current value 4, then re-associate x with 5.
x = x + 1

# Display the value currently associated with x.
print(x)
```

Code 3.3: Assignment and reassignment.

Output:

```
5
```

Code 3.4: Output from the reassignment example.

Read an assignment from right to left:

1. evaluate the expression on the right using the current values;
2. associate the name on the left with the resulting object.

3.3.1 Tracing reassignment

```
# Set the initial temperature to 20 degrees Celsius.
temperature_c = 20

# Display the initial value.
print(temperature_c)

# Add 5 degrees to the current value and store the new result.
temperature_c = temperature_c + 5

# Display the updated value.
print(temperature_c)
```

Code 3.5: Temperature reassignment traced step by step.

Step	Value associated with `temperature_c`	Action
1	20	assign initial value
2	20	print
3	25	evaluate and reassign
4	25	print

Reassignment later supports counters, loops, time-stepping algorithms, and simulations.

Do not read `=` as "is mathematically equal to." Read it as "is assigned the value of." Equality testing uses `==`, which is developed in Week 2.

3.4 Rules and conventions for variable names

A Python variable name may:

- contain letters, digits, and underscores;
- begin with a letter or underscore;
- use uppercase or lowercase letters.

A name may not:

- begin with a digit;

- contain spaces;
- be a Python keyword such as `class`, `if`, or `for`;
- be assumed to ignore capitalization.

Python is case-sensitive. The names `force`, `Force`, and `FORCE` are different.

Valid names	Invalid names
`force`	`2force` - begins with a digit
`force_n`	`beam length` - contains a space
`area_m2`	`class` - reserved keyword
`temperature_initial`	`Force` when only `force` was defined - case mismatch

The recommended style is lowercase **snake case**, in which words are separated by underscores:

```
# Beam length in metres.
beam_length_m = 2.5

# Applied force in newtons.
applied_force_n = 1000.0

# Cross-sectional area in square metres.
cross_section_area_m2 = 4.0e-4
```

Code 3.6: Recommended snake-case variable names.

3.4.1 Units in variable names

Compare an ambiguous version:

```
# Store an area value, but the unit is not visible in the name.
area = 400

# Reuse the same name after conversion, hiding the original value and unit.
area = area / 1_000_000

# Calculate stress, but the variable names do not reveal units.
stress = force / area
```

Code 3.7: Ambiguous use of a single name for two different units.

with an auditable version:

```
# Store the applied force in newtons.
force_n = 8000.0

# Store the original area in square millimetres.
area_mm2 = 400.0

# Convert the area to square metres while preserving the original value.
area_m2 = area_mm2 / 1_000_000

```

```
# Calculate normal stress in pascals: N / m^2 = Pa.
stress_pa = force_n / area_m2
```

Code 3.8: Auditable area conversion and stress calculation.

The second version preserves both area values, makes the conversion visible, and identifies the output unit.

3.5 Common Python data types

A **data type** determines what kind of object a value represents, which operations make sense, and how Python behaves when the value is used.

Type	Example	Typical Week 1 meaning
`int`	`8`	Whole-number counts, numbers of samples or bolts, indexes, and counters.
`float`	`9.81`	Measured and calculated real-valued quantities such as length, time, mass, voltage, stress, pressure, and speed.
`str`	`"steel"`	Text, labels, names, units, prompts, and messages.
`bool`	`True`	Yes/no, pass/fail, or condition states.
`NoneType`	`None`	Absence of a useful value or a result that has not been supplied.

3.5.1 Inspecting types with `type()`

```
# Store a whole-number count of bolts.
number_of_bolts = 8

# Store a measured length in metres.
beam_length_m = 2.5

# Store a material name as text.
material = "steel"

# Store a yes/no design state.
is_safe = True

# Represent the absence of a result.
result = None

# Display the type of each object in the same order as the variables above.
print(type(number_of_bolts))
print(type(beam_length_m))
print(type(material))
print(type(is_safe))
print(type(result))
```

Code 3.9: Inspecting five fundamental value types.

Typical output:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

Code 3.10: Type information returned by Python.

3.5.2 Integers and floating-point numbers

An `int` represents a whole number, such as `0`, `-3`, or `1250`. A `float` represents a finite binary approximation to a real number, such as `3.0`, `9.81`, or `4e-4`.

A useful beginner guideline is:

- use integers for counts that must be whole;
- use floats for measured or calculated engineering quantities, even when a particular value happens to look whole.

Thus, `number_of_bolts = 8` is naturally an integer, while `force_n = 8000.0` is naturally a float because force is a measured physical quantity.

3.6 Types change the meaning of operators

The same operator may perform different operations depending on the operand types.

```
# Add two integers; the result is the integer 7.
print(3 + 4)

# Join two strings; the result is the text "34".
print("3" + "4")

# Multiply two integers; the result is the integer 12.
print(3 * 4)

# Repeat the string "A" three times; the result is "AAA".
print(3 * "A")

# ERROR: Python has no meaningful rule for multiplying one string by another.
print("A" * "B")
```

Code 3.11: Predicting type-dependent operator behavior.

The first four outputs are:

```
7
34
12
AAA
```

Code 3.12: Outputs before the deliberate TypeError.

The final line raises a `TypeError`. The key lesson is that the operator symbol alone does not

determine the operation. The operand types also matter.

A string that contains digits is still text. The string `"12"` does not behave like the integer `12` or float `12.0` until it is explicitly converted.

3.7 Boolean values represent engineering questions

A Boolean value is either `True` or `False`. Comparisons produce Boolean values that can represent engineering questions.

```
# Store calculated stress in megapascals.
stress_mpa = 120.0

# Store the allowable stress in megapascals.
allowable_stress_mpa = 150.0

# Ask whether the calculated value is less than or equal to the limit.
is_within_limit = stress_mpa <= allowable_stress_mpa

# Display the answer to the engineering question.
print(is_within_limit)

# Confirm that the answer is a Boolean object.
print(type(is_within_limit))
```

Code 3.13: A Boolean variable representing a stress-limit question.

Output:

```
True
<class 'bool'>
```

Code 3.14: Boolean result and its type.

Week 2 uses Boolean conditions in branching programs. A program will then be able to take one action when a condition is true and another action when it is false.

Engineering check

A Boolean result is only as meaningful as the rule behind it. The statement “within limit” requires a clearly defined limit, compatible units, and an appropriate engineering model.

3.8 Input arrives as text

The function `input()` displays a prompt, waits for the user to type a response, and returns the response as a string.

```
# Ask the user to enter a voltage; input() returns text.
voltage = input("Enter voltage in V: ")

# Display the returned object's type.
print(type(voltage))
```

Code 3.15: Showing that input() returns a string.

Even if the user types `12`, the object is the string `"12"`. To use it in arithmetic, convert it.

```
# Ask for voltage in volts, then convert the returned string to a float.
voltage_v = float(input("Enter voltage in V: "))

# Display the numerical value and its type for verification.
print(voltage_v)
print(type(voltage_v))
```

Code 3.16: Converting a text input to a floating-point number.

Common conversion functions are:

Function	Purpose
`int(value)`	Convert to an integer when possible. For a float, the decimal part is removed rather than conventionally rounded.
`float(value)`	Convert a valid numerical string or integer to a floating-point value.
`str(value)`	Convert a value to text for labels, messages, or concatenation.

Prompts should include units. Prefer

```
# Ask explicitly for area in square millimetres before converting the input to
    float.
area_mm2 = float(input("Enter area in mm^2: "))
```

Code 3.17: A clear input prompt with the expected unit.

over the ambiguous prompt `"Enter area:  "`.

The conversion `int(3.9)` gives `3`, not `4`. It truncates the decimal part. Do not use `int()` as a substitute for a deliberate rounding rule.

3.9 A first floating-point surprise

Not every decimal fraction has an exact finite representation in binary. Consequently, a Python float may store the nearest representable value rather than the exact real number suggested by a decimal literal.

```
# Add two decimal values that do not have exact finite binary representations.
result = 0.1 + 0.2

# Display the stored approximation with Python's default representation.
print(result)

# Format the same stored value to two digits after the decimal point.
```

```
print(f"Result = {result:.2f}")
```

Code 3.18: A simple floating-point representation effect.

Typical output:

```
0.30000000000000004
Result = 0.30
```

Code 3.19: Default and formatted displays of the same float.

Formatting changes how a value is displayed; it does not necessarily change the stored value.

Key idea

Floating-point values are usually excellent approximations for introductory engineering work, but exact equality can be unreliable. Later, numerical results will be compared using tolerances.

3.10 Integrated example: stress from force and area

A force of 8,000 N acts axially on a cross-sectional area of 400 mm^2. The simplified normal-stress model is

$$\sigma = \frac{F}{A}.$$

The area conversion is

$$400\,\text{mm}^2 = \frac{400}{1{,}000{,}000}\ \text{m}^2 = 4.0 \times 10^{-4}\,\text{m}^2.$$

Therefore,

$$\sigma = \frac{8{,}000\,\text{N}}{4.0 \times 10^{-4}\,\text{m}^2} = 2.0 \times 10^{7}\,\text{Pa} = 20\,\text{MPa}.$$

```
# Store the applied axial force in newtons.
force_n = 8000.0

# Store the given cross-sectional area in square millimetres.
area_mm2 = 400.0

# Convert mm^2 to m^2; the linear factor 1000 must be squared.
area_m2 = area_mm2 / 1_000_000

# Calculate normal stress in pascals because N / m^2 = Pa.
stress_pa = force_n / area_m2

# Convert pascals to megapascals for a more readable engineering value.
stress_mpa = stress_pa / 1_000_000

# Print a descriptive heading for the calculation report.
print("AXIAL STRESS REPORT")

# Repeat a hyphen 28 times to create a visual separator.
print("-" * 28)

# Display the original force with one decimal place and the unit N.
```

```
print(f"Force = {force_n:.1f} N")

# Display the original area with one decimal place and the unit mm^2.
print(f"Area = {area_mm2:.1f} mm^2")

# Display the calculated stress with two decimal places and the unit MPa.
print(f"Stress = {stress_mpa:.2f} MPa")
```

Code 3.20: Complete annotated axial-stress report.

Expected output:

```
AXIAL STRESS REPORT
---------------------------
Force = 8000.0 N
Area = 400.0 mm^2
Stress = 20.00 MPa
```

Code 3.21: Output from the axial-stress report.

3.10.1 Assumption and interpretation

The simple formula assumes that the load is axial and that stress is uniformly distributed over the cross-section. A real component may also involve bending, stress concentrations, defects, dynamic loading, fatigue, buckling, or connection behavior. The program calculates a simplified nominal stress; it does not prove that a real component is safe.

3.10.2 Sanity trends

From $\sigma = F/A$:

- doubling force while area stays fixed should double stress;
- doubling area while force stays fixed should halve stress;
- positive force magnitude and positive area should produce positive stress magnitude;
- a unit mismatch should trigger suspicion even if Python runs without an error.

3.11 Bug hunt: name, type, unit, and output

The following deliberately faulty program combines several beginner errors.

```
# ERROR 1: the capital F creates the name Force_n, not force_n.
# ERROR 2: quotation marks store 8000 as text rather than a number.
Force_n = "8000"

# Store area as an integer in square millimetres.
area_mm2 = 400

# ERROR 3: divide by 1,000,000 for mm^2 to m^2, not by 1000.
area_m2 = area_mm2 / 1000

# ERROR 4: force_n was never defined because Python is case-sensitive.
```

```
# If the name were fixed but the string remained, division would raise TypeError.
stress_pa = force_n / area_m2

# ERROR 5: quotation marks print literal text instead of the variable's value.
print("Stress = stress_pa Pa")
```

Code 3.22: Faulty stress program with comments identifying each problem.

The errors are classified as follows:

Problem	Category	Consequence
`Force_n` versus `force_n`	Name/case error	`NameError`: the lowercase name has no assigned value.
`"8000"`	Type error	The value is text; arithmetic division is not defined between a string and float.
Divide area by 1000	Unit/logic error	Area is wrong by a factor of 1000, producing an incorrect stress.
Quoted `stress_pa`	Output/logic error	Literal characters are displayed instead of the calculated value.

A corrected version is:

```
# Store force as a floating-point number in newtons using consistent lowercase
    naming.
force_n = 8000.0

# Store area as a floating-point number in square millimetres.
area_mm2 = 400.0

# Convert square millimetres to square metres using the correct squared factor.
area_m2 = area_mm2 / 1_000_000

# Calculate stress in pascals.
stress_pa = force_n / area_m2

# Use an f-string to insert the numerical value into a labeled report.
print(f"Stress = {stress_pa:.2f} Pa")
```

Code 3.23: Corrected and annotated stress program.

3.12 Testing with simple cases before realistic ones

A plausible output is not evidence by itself. Testing is planned checking.

3.12.1 Known-case test

Choose values that are easy to calculate by hand:

$$F = 10\,\mathrm{N}, \qquad A = 2\,\mathrm{m}^2.$$

The expected stress is

$$\sigma = \frac{10}{2} = 5\,\text{Pa}.$$

```python
# Choose a force that makes the hand calculation simple.
test_force_n = 10.0

# Choose an area that makes 10 / 2 easy to verify.
test_area_m2 = 2.0

# Apply the same stress formula used in the realistic program.
test_stress_pa = test_force_n / test_area_m2

# Display the result; the expected value is exactly 5 Pa for this test case.
print(f"Test stress = {test_stress_pa:.1f} Pa")
```

Code 3.24: Known-case test for the stress formula.

3.12.2 Trend tests

Trend tests ask whether changes agree with the model:

```python
# Define a baseline force and area using compatible SI units.
force_n = 10.0
area_m2 = 2.0

# Calculate the baseline stress.
baseline_stress_pa = force_n / area_m2

# Double force while holding area fixed; stress should double.
double_force_stress_pa = (2 * force_n) / area_m2

# Double area while holding force fixed; stress should be halved.
double_area_stress_pa = force_n / (2 * area_m2)

# Display all three values for comparison.
print(f"Baseline stress = {baseline_stress_pa:.1f} Pa")
print(f"Double-force stress = {double_force_stress_pa:.1f} Pa")
print(f"Double-area stress = {double_area_stress_pa:.1f} Pa")
```

Code 3.25: Trend tests for the stress relation.

Expected output:

```
Baseline stress = 5.0 Pa
Double-force stress = 10.0 Pa
Double-area stress = 2.5 Pa
```

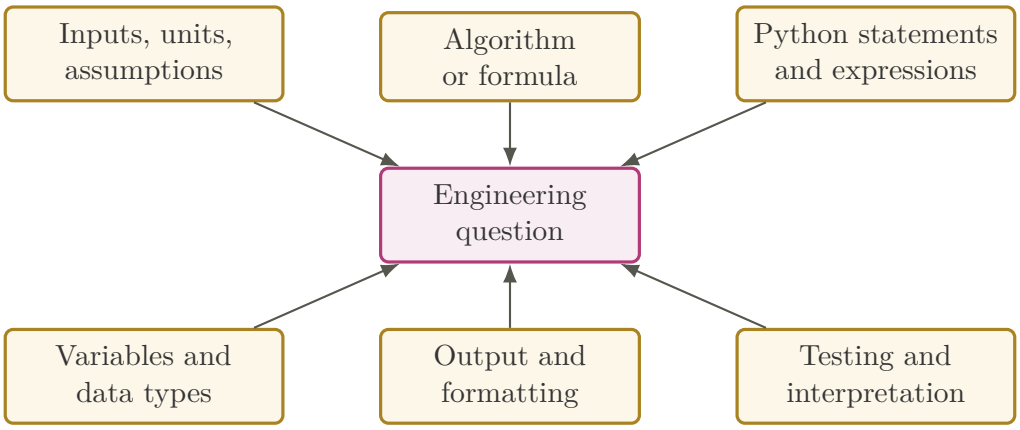
Code 3.26: Expected stress trend.

Engineering check

Testing the code and validating the model are different tasks. A program can implement F/A correctly even when the assumption of uniform axial stress is inappropriate for the real component.

3.13 The Week 1 concept map

All three lectures connect to the engineering question at the center of the work.



Code is one part of an engineering solution. A complete workflow formulates, calculates, tests, interprets, and communicates.

3.14 Lecture 3 summary

Section summary

- Variables are names associated with objects; assignment can re-associate a name with a new object.
- Python assignment `=` is not algebraic equality.
- Meaningful names and visible units make code easier to audit.
- Python is case-sensitive, and variable names must obey syntactic rules.
- Common Week 1 types are `int`, `float`, `str`, `bool`, and `NoneType`.
- Operand types affect what an operator means.
- `input()` returns a string, so numerical input normally requires conversion.
- Floats are finite binary approximations; formatted display does not alter the stored value.
- The stress example integrates units, variables, arithmetic, formatted output, assumptions, debugging, and testing.
- Known-case and trend tests provide evidence before a program is trusted.

3.15 Week 1 exit-ticket problems

Without running Python, explain the output or error for each sample.

```
# Reassign x from 3 to 5, then display 5.
x = 3
x = x + 2
```

```
print(x)

# Concatenate two strings, producing the text "32".
print("3" + "2")

# ERROR: addition between an integer and a string has no defined meaning.
print(3 + "2")

# Evaluate 4 ** 2 first, then multiply by the parenthesized sum 5.
print((2 + 3) * 4**2)
```

Code 3.27: Week 1 exit-ticket code with explanatory comments.

1. The first group prints `5`. The current value of `x` is increased by 2 and reassigned.
2. The second statement prints `32`, because `+` concatenates two strings.
3. The third statement raises a `TypeError`, because Python does not add an integer and string directly.
4. The fourth statement prints `80`: $(2 + 3) \times 4^2 = 5 \times 16 = 80$.

Before Week 2, rerun all examples from a clean Python session, complete the assigned practice from Chapters 1–2, save one error message that you encountered, and preview strings, comparisons, branching, and iteration.

4 Week 1 Synthesis, Quick Reference, and Practice

4.1 The central Week 1 workflow

The complete Week 1 message can be summarized as



A numerical output is not the end of this process. The engineer must still check that:

- the question was formulated correctly;
- the inputs and units are known;
- the model and assumptions are appropriate;
- the mathematical expression was translated accurately;
- the program was tested using independent evidence;
- the result is reported with units and interpreted in context.

4.2 Week 1 Python quick reference

Syntax	Meaning	Engineering note
`print(value)`	Display a value.	Include a quantity name and unit when reporting engineering results.
`# comment`	Human-readable note ignored by Python.	Record units, assumptions, conversions, and limitations.
`name = value`	Assignment.	Read from right to left: evaluate, then associate the name with the result.
`type(value)`	Inspect an object's type.	Useful when input or operator behavior is surprising.
`int(value)`	Convert to an integer.	Appropriate for whole counts; float-to-int conversion truncates.
`float(value)`	Convert to a floating-point number.	Commonly used for measured input returned by `input()`.
`str(value)`	Convert to text.	Useful for labels and messages.
`+ - * /`	Arithmetic operations.	Check dimensional meaning as well as numerical syntax.
`//`	Floor division.	Counts complete groups or intervals.

Syntax	Meaning	Engineering note
`%`	Remainder.	Finds leftover items or position in a periodic cycle.
`**`	Exponentiation.	Use `**`, not `^`, for powers.
`( )`	Grouping.	Make numerators, denominators, and model structure explicit.
`f"value:.2f"`	Formatted string.	Inserts a value and controls display precision; does not change the stored value.
`e notation`	Scientific notation.	`4.0e-4` means 4.0×10^{-4}.
`1_000_000`	Readable numeric literal.	Underscores improve readability without changing value.
`True, False`	Boolean values.	Represent answers to engineering conditions or design questions.
`None`	Absence of a useful value.	May indicate that a result is unavailable or not yet computed.

4.3 Engineering habits to start immediately

1. **State the question.** Write one sentence explaining what the program should determine.
2. **List inputs and outputs.** Include units.
3. **State assumptions.** Record what the simplified model includes and excludes.
4. **Predict sign, scale, and trend.** Estimate the expected behavior before running code.
5. **Use meaningful names.** Prefer `stress_mpa` to `x`.
6. **Keep units visible.** Use variable names, comments, prompts, and output labels.
7. **Translate formulas structurally.** Check every numerator, denominator, exponent, and parenthesis.
8. **Run small steps.** Inspect intermediate values when the program is unclear.
9. **Test a known case.** Use numbers for which the answer is easy to obtain independently.
10. **Test trends.** Check whether doubling or halving an input changes the output as the model predicts.
11. **Interpret the result.** State what the numerical result means and what it does not establish.
12. **Save a working version.** Preserve a correct state before making substantial changes.

4.4 Glossary

Algorithm
A finite, ordered, precise, and checkable procedure that transforms inputs into outputs.

Assignment
The Python operation that associates a name with the object produced by a right-hand-side expression.

Boolean
A value that is either `True` or `False`.

Comment
Text beginning with `#` that Python ignores but humans use for explanation.

Computation
The use of well-defined steps to transform inputs into outputs.

Data type
A category that determines what a value represents and which operations are meaningful.

Expression
A combination of values, variables, and operators that Python evaluates.

Float
A finite binary approximation to a real number.

Imperative knowledge
Knowledge of how to carry out a procedure.

Integer
A whole-number object of type `int`.

Logic error
A mistake in which the program runs but does not perform the intended calculation.

Object
A Python value with a type and associated behavior.

Program
A sequence of definitions and instructions expressed in a programming language.

Pseudocode
A structured plain-language description of an algorithm.

Runtime error
An error that occurs after program execution begins.

Statement
An instruction that tells Python to perform an action.

String
A sequence of characters used to represent text.

Syntax error
A violation of the grammatical rules of the programming language.

Unit error
A physically inconsistent use or conversion of units, often undetected by basic Python arithmetic.

Variable
A name associated with an object.

4.5 Cumulative practice

Try these problems without looking at the solutions. Predict first, then run code where requested.

4.5.1 Concept questions

1. Why is “the answer is 20” an incomplete engineering result?
2. Explain the difference between computation and programming.
3. What makes pseudocode useful before Python is written?
4. Why can a logic or unit error be more dangerous than a syntax error?
5. Why should a variable name sometimes include its unit?
6. What does it mean to say that a variable is a name associated with an object?
7. Why does `input()` commonly need to be combined with `float()` or `int()`?
8. Why does formatted output not solve a floating-point accuracy problem?

4.5.2 Code tracing

Predict the output or error.

```
# Multiplication occurs before addition.
print(4 + 3 * 2)

# Parentheses change the grouping.
print((4 + 3) * 2)

# True division retains a fractional result.
print(9 / 4)

# Floor division counts complete groups.
print(9 // 4)

# Modulo reports the remainder.
print(9 % 4)
```

Code 4.1: Practice A: arithmetic and precedence.

```
# Start with an integer value.
count = 3

# Reassign count to the current value plus two.
count = count + 2

# Display the new value and its type.
print(count)
print(type(count))

# Store digits as text rather than a number.
label = "5"

# Repeat the string twice.
```

```
print(label * 2)
```

Code 4.2: Practice B: reassignment and types.

```
# Store a numerical string returned conceptually from input.
voltage = "12"

# ERROR: division between a string and integer is not defined.
current = voltage / 4

# This line is not reached because the previous statement raises TypeError.
print(current)
```

Code 4.3: Practice C: deliberate type error.

4.5.3 Formula translation

Write Python expressions for:

1. density $\rho = m/V$;
2. average speed $v = d/t$;
3. kinetic energy $KE = \frac{1}{2}mv^2$;
4. electrical power $P = V^2/R$;
5. average of four measurements $\bar{x} = (x_1 + x_2 + x_3 + x_4)/4$.

Use variable names that include useful unit information.

4.5.4 Debugging practice

Find and repair every problem in this program.

```
# Store pressure as text instead of a number.
Pressure_kpa = "250"

# ERROR: the lowercase name pressure_kpa has not been defined.
# ERROR: multiplication would repeat text if the name mismatch were repaired only.
pressure_pa = pressure_kpa * 1000

# ERROR: quotation marks display literal text instead of the variable value.
print("Pressure = pressure_pa Pa")
```

Code 4.4: Practice D: faulty pressure-conversion program.

4.5.5 Engineering interpretation

A stress program reports `0.00002 MPa` for a moderately loaded steel bar. List at least three questions you would ask before accepting the value.

4.6 Practice solutions

4.6.1 Concept responses

1. The quantity, unit, model, assumptions, and physical interpretation are missing.
2. Computation is the broader transformation of inputs into outputs by defined steps; programming is the expression of such steps in an executable language.
3. Pseudocode clarifies inputs, order, conversions, and checks before syntax becomes a distraction.
4. Syntax errors are visible and stop execution; logic or unit errors may produce convincing but incorrect outputs.
5. A unit-bearing name makes conversions auditable and reduces accidental mixing of unit systems.
6. The name refers to an object; reassignment can associate the same name with a different object.
7. `input()` returns text, while arithmetic generally requires numerical types.
8. Formatting changes only the displayed representation. It does not alter the stored approximation or validate the calculation.

4.6.2 Code-tracing solutions

Practice A outputs `10`, `14`, `2.25`, `2`, and `1`.

Practice B outputs `5`, `<class 'int'>`, and `55`. The final result is string repetition, not the number 10.

Practice C raises a `TypeError` at the division because `voltage` is a string.

4.6.3 Formula translations

One clear set of answers is:

```
# Calculate density in kg/m^3 from mass in kg and volume in m^3.
density_kg_per_m3 = mass_kg / volume_m3

# Calculate average speed in m/s from distance in m and time in s.
average_speed_m_per_s = distance_m / time_s

# Calculate kinetic energy in joules using KE = 0.5 m v^2.
kinetic_energy_j = 0.5 * mass_kg * velocity_m_per_s**2

# Calculate electrical power in watts using P = V^2 / R.
power_w = voltage_v**2 / resistance_ohm

# Add all four measurements before dividing by four.
average = (x1 + x2 + x3 + x4) / 4
```

Code 4.5: Annotated solutions to the formula-translation practice.

4.6.4 Debugging solution

```python
# Store pressure as a floating-point number in kilopascals.
pressure_kpa = 250.0

# Convert kilopascals to pascals using 1 kPa = 1000 Pa.
pressure_pa = pressure_kpa * 1000

# Insert the variable value into a labeled output string.
print(f"Pressure = {pressure_pa:.1f} Pa")
```

Code 4.6: Corrected and annotated pressure-conversion program.

4.6.5 Interpretation response

Useful questions include:

- Was force expressed in newtons?
- Was area expressed in square metres before evaluating F/A?
- Was the conversion from pascals to megapascals performed correctly?
- Is the force and cross-sectional area realistic for the component?
- Does a simple hand calculation give the same order of magnitude?
- Does doubling force double the reported stress?
- Is the simplified uniform axial-stress model appropriate?

Key idea

Welcome to computational engineering: ask precise questions, write clear instructions, test the result, and explain what it means.

Source Basis and Suggested Reading

These notes were developed to accompany the complete Week 1 lecture slides and preserve their organization, terminology, examples, and engineering emphasis. The principal sources are:

1. Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026.
2. Yanwei Wang, *Programming for Engineers with Python*, working course-manuscript draft, Chapters 1–2 and associated instructor notes.
3. John V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Chapters 1–2.
4. Yanwei Wang, *Programming for Engineers - Week 1: Course Launch, Computation, Python Basics, Variables, and Types*, lecture-slide set, School of Engineering, Nazarbayev University, Fall 2026.

Primary preparation for students

Read the assigned sections and run every short example. For each example:

1. predict the output or error;
2. run the code;
3. change one value or unit;
4. predict the effect of the change;
5. rerun the program from a clean session;
6. explain the result and the evidence used to check it.